

Experiment No. 5

Spam Email Classification using Support Vector Machine (SVM)

Aim

Write a Python script to train a model (SVM) to classify emails as spam or not spam.

Theory

1. Spam Email Classification

Spam classification is a **binary classification problem** in NLP where:

- **Spam (1)** → Unwanted/advertisement/fraud emails
- **Ham (0)** → Legitimate emails

Machine learning models learn patterns in text (keywords, frequency, structure) to classify emails.

2. Text Preprocessing

Before applying ML, raw email text must be cleaned:

- **Lowercasing** → Ensures uniformity
- **Stopword Removal** → Removes common words like *the, is, and*
- **Tokenization** → Splitting text into words
- **Punctuation Removal** → Removes noise

This step improves model accuracy and reduces dimensionality.

3. Feature Extraction (Vectorization)

Machine learning models cannot understand text directly, so we convert it into numerical vectors.

TF-IDF

- Assigns importance to words
 - Reduces weight of common words
-

4. Support Vector Machine (SVM)

SVM is a **supervised machine learning algorithm** used for classification.

Working Principle

- Finds an optimal **hyperplane** that separates two classes
- Maximizes the **margin** between classes
- Uses **support vectors** (critical boundary points)

Mathematical Concept

For classification:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
- x = input features
- b = bias

Decision rule:

- $w \cdot x + b \geq 0 \rightarrow$ Spam
 - $w \cdot x + b < 0 \rightarrow$ Not Spam
-

Algorithm / Procedure

1. Import required libraries
2. Load dataset (spam/ham emails)
3. Preprocess text (lowercase, remove stopwords, etc.)
4. Convert text to numerical features using **TF-IDF**
5. Split dataset into:
 - Training set
 - Testing set
6. Train **SVM classifier**
7. Predict on test data
8. Evaluate performance using:
 - Accuracy
 - Confusion Matrix
 - Precision & Recall

Program and output

Sample Output (Expected)

Accuracy: 0.83

Confusion Matrix:

```
[[2 0]
 [1 1]]
```

Classification Report:

	precision	recall	f1-score	support
Ham	1.00	1.00	1.00	2
Spam	1.00	0.50	0.67	2

Conclusion

In this experiment, we implemented a **spam email classifier using SVM**.

We performed text preprocessing, feature extraction using TF-IDF, and trained a classification model. The SVM algorithm effectively separated spam and non-spam emails by finding an optimal decision boundary.

This experiment demonstrates how **machine learning + NLP techniques** can be used in real-world applications like email filtering.

Experiment No. 6

Word Embeddings using Word2Vec and GloVe for Text Similarity Analysis

Aim

Write a Python script to train Word2Vec embeddings using Gensim and load pre-trained GloVe embeddings for text similarity analysis.

Theory

1. Word Embeddings

Word embeddings are dense vector representations of words used in NLP where:

- Each word is represented as a numerical vector
- Similar words have similar vector representations

Unlike traditional methods (BoW, TF-IDF), embeddings capture **semantic meaning and relationships** between words.

2. Word2Vec (Gensim)

Word2Vec is a neural network-based model used to learn word embeddings from text data.

Types of Word2Vec:

- **CBOW (Continuous Bag of Words)** → Predicts a word from its context
- **Skip-Gram** → Predicts context words from a target word

Working Principle:

- Words appearing in similar contexts have similar meanings
- Uses a shallow neural network to learn embeddings

Advantages:

- Captures semantic relationships
 - Efficient for large datasets
-

3. GloVe (Global Vectors for Word Representation)

GloVe is a **pre-trained embedding model** developed using global word co-occurrence statistics.

Key Features:

- Combines statistical and predictive approaches
- Trained on large corpora (Wikipedia, Common Crawl)
- Provides high-quality embeddings

Unlike Word2Vec (trained locally), GloVe captures **global relationships** between words.

4. Text Similarity using Embeddings

Text similarity measures how similar two words/sentences are.

Method:

- Convert words into vectors
- Use similarity metrics like **Cosine Similarity**

Formula:

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- A, B = word vectors

Higher value → more similar words

Algorithm / Procedure

- Import required libraries
- Prepare a sample text dataset
- Tokenize text into sentences and words
- Train Word2Vec model using Gensim
- Extract word vectors
- Load pre-trained GloVe embeddings from file
- Create a dictionary of word vectors
- Compute similarity between words using:
 - Word2Vec model
 - GloVe embeddings
- Compare similarity results

Code and Output

Sample Output (Expected)

Word2Vec Similarity (learning, ai): 0.72

GloVe Similarity (learning, ai): 0.65

Conclusion

In this experiment, we implemented **Word2Vec embeddings using Gensim** and loaded **pre-trained GloVe embeddings** for text similarity analysis.

Word2Vec learned word relationships from the given dataset, while GloVe provided rich pre-trained representations. Cosine similarity was used to measure semantic similarity between words.

This experiment demonstrates how **word embeddings capture semantic meaning** and are useful in tasks like search engines, recommendation systems, and NLP applications.

Experiment No. 7

Sentiment Analysis using Naïve Bayes and Logistic Regression

Aim

Write a Python script to train Naïve Bayes and Logistic Regression classifiers for sentiment analysis.

Theory

1. Sentiment Analysis

Sentiment analysis is a Natural Language Processing (NLP) task used to determine the emotional tone of a piece of text. It classifies text into categories such as positive, negative, or neutral. This technique is widely used in applications like product reviews, social media analysis, and customer feedback systems. The main objective is to automatically understand human opinions and sentiments from textual data using machine learning models.

2. Text Preprocessing

Before applying any machine learning algorithm, the raw text must be cleaned and standardized. Text preprocessing includes converting all characters to lowercase to maintain uniformity, removing stopwords that do not contribute significant meaning, eliminating punctuation marks, and tokenizing the text into individual words. These steps help in reducing noise and improving the efficiency and accuracy of the classification models by ensuring that only meaningful information is used for training.

3. Feature Extraction (TF-IDF)

Machine learning models require numerical input, so textual data must be transformed into vectors. TF-IDF (Term Frequency–Inverse Document Frequency) is a commonly used technique that assigns importance to words based on their frequency in a document and their rarity across all documents. Words that appear frequently in a specific document but are rare across the dataset are given higher importance. This approach improves the quality of features compared to simple frequency-based methods and enhances model performance.

4. Naïve Bayes Classifier

Naïve Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes that all features are independent of each other, which is known as the “naïve” assumption. Despite this simplification, it performs very well in text classification tasks. The classifier calculates the probability of a text belonging to a particular class and selects the class with the highest probability. It is computationally efficient and works effectively with high-dimensional data such as text.

Mathematical Concept:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Where:

- $P(C | X)$ = Probability of class given data
- $P(X | C)$ = Likelihood
- $P(C)$ = Prior probability
- $P(X)$ = Evidence

5. Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for binary classification problems. It predicts the probability of a class using the sigmoid function, which maps any real-valued number into a value between 0 and 1. Based on this probability, the model classifies the input into one of the two categories. Unlike Naïve Bayes, Logistic Regression does not assume feature independence and can model more complex relationships between features.

Mathematical Concept:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- $z = wx + b$
- Output is a probability between 0 and 1

Decision Rule:

- If probability $\geq 0.5 \rightarrow$ Positive sentiment
- If probability $< 0.5 \rightarrow$ Negative sentiment

Algorithm / Procedure

- Import required libraries
- Load or create a labeled sentiment dataset
- Preprocess the text (lowercase, remove stopwords, etc.)
- Convert text into numerical form using TF-IDF
- Split dataset into training and testing sets
- Train Naïve Bayes classifier
- Train Logistic Regression classifier
- Predict sentiment on test data
- Evaluate models using accuracy and classification report
- Compare the performance of both models

Code and Output

Sample Output (Expected)

Naïve Bayes Accuracy: 0.83

Logistic Regression Accuracy: 1.00

Conclusion

In this experiment, we implemented sentiment analysis using both Naïve Bayes and Logistic Regression classifiers. The text data was preprocessed and converted into numerical form using TF-IDF. Both models were trained and evaluated on the dataset. It was observed that while Naïve Bayes is faster and effective for basic text classification, Logistic Regression can achieve higher accuracy due to its ability to model more complex relationships.

This experiment highlights the importance of choosing appropriate machine learning models for NLP tasks and demonstrates how sentiment analysis can be applied in real-world scenarios such as customer feedback analysis and opinion mining.

Experiment No. 8

Topic Modeling using Latent Dirichlet Allocation (LDA)

Aim

Write a Python script to implement LDA to discover hidden topics in a set of documents.

Theory

1. Topic Modeling

Topic modeling is an unsupervised Natural Language Processing (NLP) technique used to identify hidden themes or topics within a collection of documents. Instead of manually labeling data, topic modeling automatically discovers patterns of word usage and groups related words into topics. Each document is then represented as a mixture of multiple topics, and each topic is represented as a distribution of words. This makes it highly useful for organizing, summarizing, and understanding large text datasets such as news articles or research papers.

2. Latent Dirichlet Allocation (LDA)

Latent Dirichlet Allocation (LDA) is a probabilistic model used for topic modeling. It assumes that documents are generated from a mixture of topics, where each topic is a probability distribution over words. The term “latent” refers to hidden topics, and “Dirichlet” refers to the probability distributions used in the model. LDA works by reverse-engineering this process: given a set of documents, it tries to infer the underlying topics and how strongly each document is associated with those topics.

3. Working Principle of LDA

LDA assumes that each document contains multiple topics in different proportions. For each word in a document, the model assigns a topic based on probability distributions. Over many iterations, LDA adjusts these probabilities to maximize the likelihood of observing the given data. As a result, words that frequently occur together tend to form a topic. For example, words like “government,” “election,” and “policy” may form a political topic, while “AI,” “data,” and “learning” may form a technology-related topic.

4. Dirichlet Distribution

LDA uses Dirichlet distributions to model the distribution of topics in documents and words in topics. A Dirichlet distribution ensures that the probabilities are positive and sum to one. It acts as a prior that controls how topics are distributed across documents and how words are distributed within topics. This helps in maintaining a balanced and meaningful representation of topics.

5. Applications of LDA

LDA is widely used in various real-world applications such as document clustering, recommendation systems, information retrieval, and text summarization. It helps in extracting meaningful insights from large volumes of unstructured text data and is commonly used in analyzing customer reviews, social media content, and research articles.

Algorithm / Procedure

- Import required libraries
- Prepare a collection of text documents

- Preprocess text (lowercase, remove stopwords, tokenize)
- Create a dictionary of words
- Convert documents into a document-term matrix
- Apply LDA model to extract topics
- Define number of topics
- Train the LDA model
- Display topics with important words
- Analyze topic distribution in documents

Sample Output (Expected)

Topics discovered:

Topic 0: 0.2*"machine" + 0.2*"learning" + 0.1*"ai" + ...

Topic 1: 0.3*"government" + 0.2*"politics" + ...

Document Topic Distribution:

Document 0: [(0, 0.85), (1, 0.15)]

Document 1: [(0, 0.90), (1, 0.10)]

Document 2: [(1, 0.88), (0, 0.12)]

Conclusion

In this experiment, we implemented Latent Dirichlet Allocation (LDA) to discover hidden topics in a collection of documents. The text data was preprocessed and converted into a document-term matrix, and the LDA model was applied to extract meaningful topics. The results showed that LDA successfully grouped related words into topics and identified the dominant topics in each document. This experiment demonstrates how unsupervised learning techniques like LDA can be used to analyze large text datasets and uncover hidden patterns, making it useful in applications such as document classification, recommendation systems, and content analysis.

Experiment No. 10

Text Summarization using NLP Techniques and Transformers

Aim

Write a Python script to implement text summarization using NLP techniques and transformers.

Theory

1. Text Summarization

Text summarization is a Natural Language Processing (NLP) task that aims to generate a concise and meaningful summary of a longer text while preserving its important information. It helps in reducing large volumes of text into shorter versions, making it easier to understand and analyze content quickly. Text summarization is widely used in applications such as news aggregation, document analysis, and search engines.

2. Types of Summarization

There are two main approaches to text summarization: extractive and abstractive. Extractive summarization selects important sentences directly from the original text without modifying them, whereas abstractive summarization generates new sentences that capture the meaning of the original text. Abstractive methods are more advanced and closer to human-like summarization, as they can paraphrase and restructure information.

3. NLP-Based Summarization Techniques

Traditional NLP-based summarization methods rely on statistical and linguistic features such as word frequency, sentence importance, and position of sentences in the text. Techniques like frequency-based scoring and TF-IDF are used to identify key sentences. Although these methods are simple and efficient, they often fail to capture deep semantic meaning and context.

4. Transformers in NLP

Transformers are deep learning models that have revolutionized NLP by enabling better understanding of context and relationships in text. They use mechanisms like self-attention to process entire sentences at once rather than sequentially. Models such as BERT, GPT, and T5 are based on transformer architecture and are widely used for tasks like translation, summarization, and question answering.

5. Abstractive Summarization using Transformers

Transformer-based models such as T5 and BART are commonly used for abstractive summarization. These models are pre-trained on large datasets and fine-tuned for summarization tasks. They generate summaries by understanding the meaning of the input text rather than simply extracting sentences. This results in more coherent, natural, and context-aware summaries compared to traditional methods.

Algorithm / Procedure

- Import required libraries
- Load a pre-trained transformer model for summarization
- Input the text to be summarized

- Preprocess the text if required
- Tokenize the input text
- Generate summary using the model
- Decode the generated tokens into readable text
- Display the summarized output

Code and output

Sample text = ""

Artificial Intelligence is transforming industries across the world.

It enables machines to learn from data and make decisions.

AI is used in healthcare, finance, education, and many other fields.

However, it also raises ethical concerns regarding privacy and job displacement.

""

Sample Output (Expected)

Summary:

Artificial Intelligence is transforming industries and enabling machines to learn from data.

It is widely used across various sectors but raises ethical concerns.

Conclusion

In this experiment, we implemented text summarization using both traditional NLP concepts and transformer-based models. While basic NLP techniques rely on statistical methods to extract key sentences, transformer models like those used in this experiment generate more meaningful and context-aware summaries. The use of pre-trained models simplifies implementation and improves performance significantly.

This experiment demonstrates how modern NLP techniques can efficiently process and summarize large textual data, making them highly useful in real-world applications such as news summarization, document analysis, and content recommendation systems.